3강. 코드 품질 자동화

husky · lint-staged · commitlint

AI가 코드를 만든다.
품질은 누가 보장하나?

Claude가 생성한 코드에 ESLint 에러가 있어도 알림이 없다.

`npm run lint` 실행을 잊으면 그냥 커밋된다.

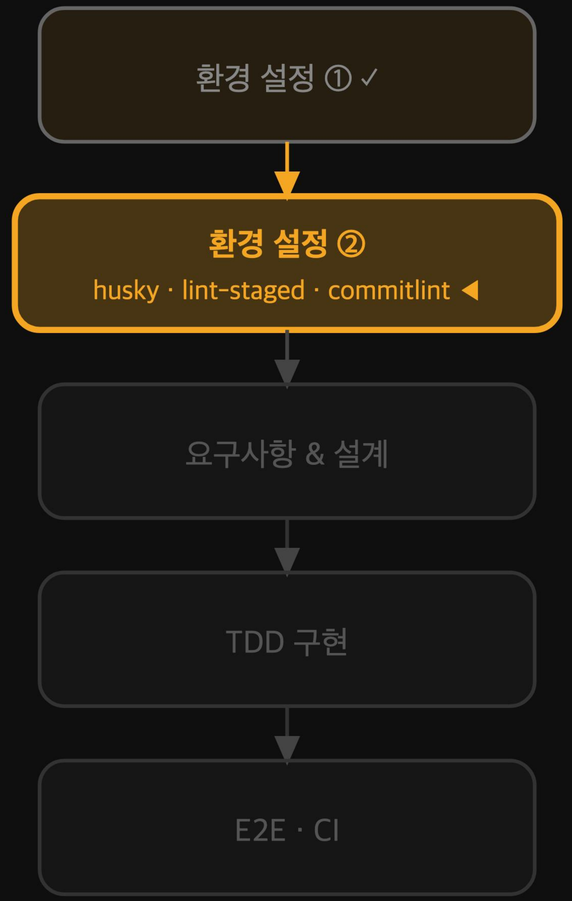
사람이 기억해야 할 것은 시스템에 위임한다.

지금 여기 있습니다

환경 설정 ② husky · lint-staged · commitlint

이전: 2강에서 CLAUDE.md 완료

다음: 요구사항 & 설계 단계 진입



이번 강의 목표

Git Hook 원리

pre-commit, commit-msg의 실행 메커니즘과 차단 원리를 설명할 수 있다

도구 설정

husky · lint-staged · commitlint를 설치하고 설정할 수 있다

Conventional Commits

규칙 구조와 목적을 설명하고 CLAUDE.md에 추가할 수 있다

Git Hook — 커밋에 끼어드는 스크립트

```
git commit
    │
    ▼
.git/hooks/pre-commit 실행
    ├── exit 0 → 커밋 진행
    └── exit 1 → 커밋 차단
```

pre-commit — 커밋 객체 생성 전. 코드 품질 검사

commit-msg — 메시지 기록 전. 형식 검사

Husky — 훅을 팀이 공유하는 방법

```
# 기존: git 추적 안 됨 — 팀원 클론 시 훅 없음
.git/hooks/pre-commit

# Husky: git 추적됨
.husky/pre-commit
```

`npm install`만 하면 팀 전체에 동일한 훅이 **자동 적용**된다

lint-staged — staged 파일만 빠르게

```
pre-commit → lint-staged: staged .ts, .tsx 파일만
    ├── ESLint --fix   (자동 수정 가능한 것은 고침)
    ├── Prettier --write (포맷 정리)
    ├── 문제 없음 → 커밋 완료
    └── 자동 수정 불가 → 커밋 차단
```

전체 lint 대신 **방금 수정한 파일만** 검사 — 빠르고 정확하다

Conventional Commits

```
feat(tag): 노트에 태그 추가 기능 구현
fix(tag): 중복 태그 추가 시 에러 수정
test(tag): addTag 유효성 검사 테스트 추가
refactor(tagInput): TagInput 컴포넌트 책임 분리
```

타입	용도
feat	새로운 기능
fix	버그 수정
test	테스트 추가/수정
refactor	동작 변경 없는 구조 개선
chore	빌드, 설정, 패키지 변경

commitlint + 전체 파이프라인

```
git commit -m "feat(tag): 태그 추가"
    ▼
[1] pre-commit → lint-staged → 코드 검사
    ▼
[2] commit-msg → commitlint → 메시지 형식 검사
    ▼
통과 시 커밋 완료
```

commitlint: 형식이 틀리면 커밋 차단 — CLAUDE.md에 규칙 추가 시 AI도 준수

실습

husky + lint-staged

설치 및 pre-commit 훅 설정

commitlint

설치 및 commit-msg 훅 설정

동작 테스트

lint 에러 / 잘못된 메시지 차단 확인

CLAUDE.md 업데이트

Conventional Commits 규칙 추가